



PLAYWRIGHT LOCATORS & SELECTOR STRATEGIES

Find elements reliably and write resilient automation

PW

1. LOCATOR STRATEGIES IN PLAYWRIGHT

Strategy	How to use (Example)	Description	Best Use
getByRole 	<code>page.getByRole('button', { name: 'Submit' });</code>	Finds elements by ARIA role and accessible name.	✓ Preferred for accessibility and user-friendly elements.
getByText 	<code>page.getByText('Welcome back');</code>	Finds elements by exact text content.	✓ Good for static text visible to users.
getByLabel 	<code>page.getByLabel('Username');</code>	Finds form elements by the associated label text.	✓ Great for form fields (input, textarea, etc.).
getByPlaceholder 	<code>page.getByPlaceholder('Enter email');</code>	Finds input elements by placeholder attribute.	⚠ Use when label is not available.
getByTestId 	<code>page.getByTestId('login-btn');</code>	Finds elements by data-testid attribute.	✓ Best for dynamic UI and stable testing.
getTitle 	<code>page.getByTitle('Dashboard');</code>	Finds elements by title attribute.	✓ Use for icons, links or elements with title.
CSS Selector 	<code>page.locator('div.container > button');</code>	Finds elements using CSS selectors.	✓ Use for complex structures (last resort).
XPath 	<code>page.locator("//button[text()='Save']");</code>	Finds elements using XPath expressions.	⚠ Use when CSS is not sufficient.

2. EXAMPLES IN ACTION

By Role	<code>await page.getByRole('button', { name: 'Login' }).click();</code>
By Text	<code>await page.getByText('Thank you').isVisible();</code>
By Label	<code>await page.getByLabel('Email address').fill('test@mail.com');</code>
By Placeholder	<code>await page.getByPlaceholder('Search...').fill('Playwright');</code>
By Test Id	<code>await page.getByTestId('submit-btn').click();</code>
By Title	<code>await page.getByTitle('Download file').click();</code>
By CSS	<code>await page.locator('ul.menu > li.active > a').click();</code>
By XPath	<code>await page.locator("//input[@type='checkbox' and @name='agree']").check();</code>

3. BEST PRACTICES

- ✓ **Prefer user-centric locators**
Use `getByRole`, `getByLabel`, `getByText` as they represent how users interact with the page.
- ✓ **Use data-testid for dynamic UI**
Add `data-testid` for elements that don't have stable text, labels or roles.
- ⚠ **Avoid CSS/XPath**
They are brittle and prone to break with UI changes. Use only when necessary.
- 🎯 **Be specific**
Make locators as specific as possible to avoid matching multiple elements.
- 🔄 **Keep locators maintainable**
Avoid deep nesting and overly complex selectors.

4. TIPS FOR RESILIENT LOCATORS

 Prefer stable attributes Use role, label, text, placeholder, testid.	 Use partial text when needed <code>page.getByText(/log in/)</code> (for partial match)	 Combine locators <code>page.getByRole('row').filter({ hasText: 'John' })</code>	 Use locator filters <code>.filter(), .first(), .last(), .nth(), .has(), .hasText()</code>	 Leverage auto-wait Playwright auto-waits for visibility, stability and receives events.
---	---	--	---	--

5. ADVANCED: COMBINING & CHAINING LOCATORS

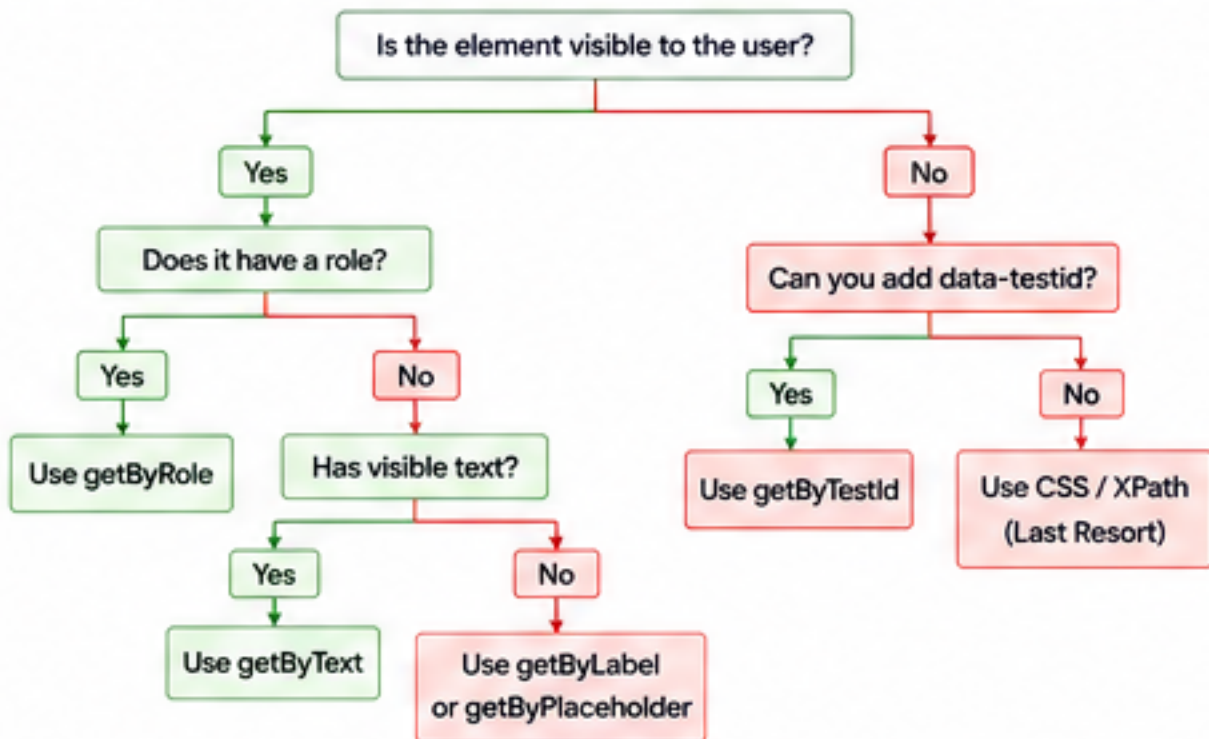
Using filter() with hasText

```
const row = page.getByRole('row')
  .filter({ hasText: 'John Doe' });
await row.getByRole('button', { name: 'Edit' }).click();
```

Chaining Locators

```
await page.getByRole('table')
  .getByRole('row').nth(2)
  .getByRole('button', { name: 'Delete' }).click();
```

6. WHEN TO USE WHICH?



7. QUICK REFERENCE

Strategy	Priority
<code>getByRole</code>	★★★★★
<code>getByLabel</code>	★★★★★
<code>getByText</code>	★★★★☆
<code>getByTestId</code>	★★★★☆
<code>getByPlaceholder</code>	★★★★☆
<code>getTitle</code>	★★★☆☆
<code>CSS Selector</code>	★★☆☆☆
<code>XPath</code>	★☆☆☆☆
Top = Recommended Bottom = Use only if needed	